Answer Convergence as a Signal for Early Stopping in Reasoning

Xin Liu and **Lu Wang**
Computer Science and Engineering
University of Michigan
Ann Arbor, MI
{liuxincs, wangluxy}@umich.edu

arXiv:2506.02536v2 [cs.CL] 29 Sep 2025

Abstract

Chain-of-thought (CoT) prompting enhances reasoning in large language models (LLMs) but often leads to verbose and redundant outputs, thus increasing inference cost. We hypothesize that many reasoning steps are unnecessary for producing correct answers. To investigate this, we start with a systematic study to examine what is the minimum reasoning required for a model to reach a stable decision. We find that on reasoning tasks like math, models typically converge to their final answers after 60% of the reasoning steps, suggesting substantial redundancy in the remaining content. Based on these insights, we propose three inference-time strategies to improve efficiency: (1) early stopping via answer consistency, (2) boosting the probability of generating end-of-reasoning signals, and (3) a supervised method that learns when to stop based on internal activations. Experiments across five benchmarks and five open-weights LLMs show that our methods significantly reduce token usage with little or no accuracy drop. In particular, on NaturalQuestions, Answer Consistency reduces tokens by over 40% while further improving accuracy. Our work underscores the importance of cost-effective reasoning methods that operate at inference time, offering practical benefits for real-world applications.[1]

1 Introduction

Large language models (LLMs) exhibit strong reasoning capabilities through step-by-step generation, known as chain-of-thought (CoT) reasoning (Wei et al., 2022; Cobbe et al., 2021; Fang et al., 2024). However, this approach often leads to unnecessarily long and verbose reasoning traces, resulting in high inference cost and latency. This *overthinking phenomenon* has become a practical bottleneck, especially in real-time or resource-constrained scenarios (Sui et al., 2025).

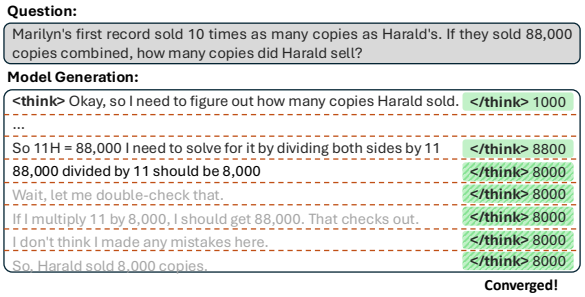

Figure 1: R1-Qwen-32B converges early on a GSM8K example, suggesting that later steps could be skipped.

To improve reasoning efficiency, recent studies have introduced methods that allow models to generate accurate answers with fewer steps. These include reinforcement learning with length-aware rewards (Luo et al., 2025; Hou et al., 2025), fine-tuning on variable-length CoT traces (Han et al., 2024; Xia et al., 2025), and prompt-based approaches that request concise reasoning (Xu et al., 2025; Nayab et al., 2024; Han et al., 2024). These methods typically require retraining on curated data or task-specific prompt design. In contrast, we explore inference-time techniques that improve efficiency without sacrificing accuracy. Particularly, we hypothesize that LLMs often internally converge on an answer before completing the full reasoning trace, an insight we formalize as *answer convergence*. Recognizing such convergence can enable more efficient inference by allowing early stopping without sacrificing accuracy.

To investigate this, we start with a reasoning model early-stopping study that systematically truncates explicit CoTs to assess when the model's answer generated answer converges, i.e., when the answer remains unchanged despite additional reasoning steps. Our experiments reveal that models often converge well before completing the full reasoning chain, suggesting substantial redundancy

[1] Additional details about the project are available on its Hugging Face page: https://huggingface.co/spaces/launch/reasoning_earlystop

and highlighting the potential for improving efficiency through early stopping. Figure 1 shows an example where the model answer converges early despite receiving only partial reasoning, indicating that the remaining steps contribute little to the final prediction. As shown in Figure 2, this pattern holds across five datasets, even on GPQA, a challenging dataset, many examples converge early.

Motivated by this finding, we propose inference-time strategies to dynamically truncate explicit reasoning based on the observation that models often reach converged answers early: (1) Early stopping via **Answer Consistency**, which halts generation when consecutive reasoning chunks yield identical answers; (2) **Think Token Adjustment**, which encourages models to signal early termination explicitly; and (3) a supervised approach, **Learn-to-Stop**, which utilizes internal activations to predict optimal stopping points. Our methods are *model-agnostic*, require no additional training or LM modification, and significantly reduce inference cost without sacrificing accuracy. Importantly, this method *does not require ground-truth answer labels*, relying only on self-consistency signals during inference.

We evaluate our methods on five reasoning benchmarks using five open-weights LLMs. Results show that early stopping strategies substantially reduce token usage. The supervised approach (*Learn-to-Stop*) consistently maintains accuracy across both simple and complex tasks, while the two unsupervised methods offer strong efficiency gains particularly on simpler datasets. Specifically, (1) **Performance vs. token length**: *Learn-to-Stop* cuts up to 48% of tokens on NQ with QwQ-32B, sometimes even improving accuracy, suggesting that excessive reasoning may introduce unnecessary noise. (2) **Supervised vs. unsupervised**: Unsupervised methods work well on NQ and GSM8K, while the supervised approach generalizes better to harder tasks like MATH-500 and GPQA.

2 Related Work

Prior work on improving CoT reasoning efficiency generally falls into four categories. Reinforcement learning-based approaches encourage brevity by introducing length-aware reward functions or adapting reasoning length to problem difficulty (Luo et al., 2025; Hou et al., 2025; Shen et al., 2025). Supervised fine-tuning methods compress reasoning traces by skipping unimportant tokens (Xia et al., 2025), enforcing token budgets (Han et al., 2024), or self-training models on shorter rationales through best-of-N sampling and few-shot prompts (Munkhbat et al., 2025). Meanwhile, prompt-based techniques guide models to generate minimal intermediate steps (Xu et al., 2025; Aytes et al., 2025), or use routing mechanisms to dynamically choose efficient reasoning paths or backbones (Cheng et al., 2025; Ong et al., 2024).

In contrast to these methods, which rely on training-time optimization on carefully designed datasets or application-specific prompt design, our work provides lightweight inference-time methods. By identifying internal answer convergence, it enables dynamic early stopping without requiring additional training, model modifications, large-scale labeled data, or task-specific prompts.

Concurrent to our research, Yang et al. (2025) also address the overthinking problem with a dynamic early exit strategy (DEER). Whereas their method relies on the model's internal confidence in a trial answer , our work proposes complementary approaches based on output consistency and learned stopping signals from activations.

3 Preliminary

We investigate whether all steps of a reasoning chain are necessary for the model to converge on its predicted answer, and whether later steps can be omitted without affecting the decision. To this end, we first split the CoT into sentence-level chunks, treating each sentence as a distinct reasoning step, using the NLTK tokenizer (Bird et al., 2009). These chunks are then incrementally concatenated (e.g., `chunk1`, `chunk1+chunk2`, etc.), each followed by an end-of-reasoning token `</think>`. The model is prompted to generate an answer from each partial chain via greedy decoding. By tracking when the model's prediction remains unchanged across successive reasoning steps, we identify the earliest point of answer convergence, approximating *the minimum explicit CoT reasoning required for the model to reach a stable decision*.[2]

We apply this protocol to five tasks with varying levels of reasoning: **NaturalQuestions (NQ)** (Kwiatkowski et al., 2019), **GSM8K** (Cobbe et al., 2021), **MATH-500** (Lightman et al.,

[2]We use "reasoning" in this paper primarily to refer to explicit chain-of-thought (CoT) reasoning. While models may also perform latent reasoning internally (Geiping et al., 2025; Hao et al., 2024), our focus is on improving the efficiency of the explicit reasoning process.

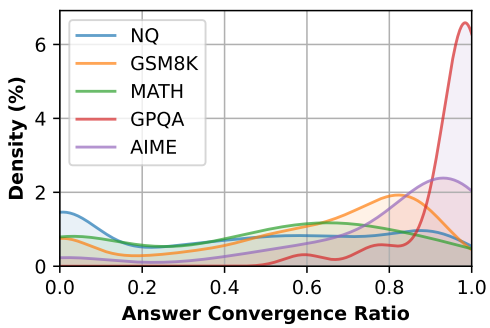

Figure 2: Distribution of Answer Convergence Ratios (ACRs) across tasks. Models often converge on their answers early, suggesting that many generated steps might be redundant from the model's perspective.

2024), **GPQA-Diamond** (Rein et al., 2023), and **AIME'24**[3]. NQ involves minimal reasoning as an information-seeking task, while the others are math and logic benchmarks of increasing difficulty, with AIME'24 containing the most advanced problems. Experiments are conducted using the R1-distilled Qwen-32B model (DeepSeek-AI, 2025)[4]. To quantify when the model converges, we define the **Answer Convergence Ratio (ACR)** as the proportion of the explicit reasoning chain required before the predicted answer remains unchanged. Specifically, we first split each CoT trace into sentence-level chunks using the NLTK tokenizer, treating each sentence as one reasoning step. The total number of chunks for a given example defines the original number of steps in the full reasoning chain. For each instance, we incrementally reveal the reasoning steps and monitor the model's output. We then identify the earliest chunk at which the predicted answer remains consistent through to the end. For example, if the answer stabilizes at the 7th step out of 10 total chunks, the ACR is 0.7.

Figure 2 displays the distribution of ACRs across the five tasks. We observe that the model often converges early, suggesting that many steps are unnecessary. The ACR distribution peaks near 0.0 for NQ, which means nearly no reasoning is needed for the model to come up with its final answer, around 0.8 for GSM8K and MATH-500, and near 0.9 for GPQA and AIME'24, mirroring the increasing reasoning loads. These results suggest that early stopping is feasible and can reduce inference-time costs without affecting answer quality.

[3] https://huggingface.co/datasets/HuggingFaceH4/aime_2024

[4] Results for various models are provided in Appendix A.1.

4 Early Stopping at Inference Time

We propose three methods to improve reasoning efficiency: two unsupervised approaches based on answer consistency (§4.1) and decoding signals (§4.2) and one supervised method (§4.3) that predicts when to stop reasoning without retraining the LLM or modifying its parameters.

4.1 Detecting Answer Consistency

Since LLMs often reach converged answers before completing the full reasoning chain, we introduce an unsupervised stopping criterion based on output consistency. During decoding, we monitor the model's outputs and append the `</think>` token at predicted natural sentence boundaries, prompting it to produce an answer via greedy decoding. If the same answer is produced for a fixed number k of consecutive chunks, we consider the reasoning converged and terminate further generation.

4.2 Think Token Adjustment

During decoding, the model uses the `</think>` token to indicate the end of reasoning. Ideally, if the model has reached the final answer, it should generate this token early. However, we observe that while `</think>` often ranks among the top 10 candidates after sentence boundaries, the model tends to prefer tokens like `wait`, `or`, or `but`, which unnecessarily prolong reasoning.

To address this, we boost the probability of `</think>` during decoding by applying a linear logit transformation[5]:

$$y_{t^*} \leftarrow y_{t^*} + \alpha \cdot (\max(\mathbf{y}) - \frac{1}{|\mathbf{y}|} \sum_i y_i), \quad (1)$$

where y_{t^*} is the logit of the `</think>` token, α controls the boost strength, $\mathbf{y}$ denotes all vocabulary logits, y_i denotes the logit of i-th token in the vocabulary. This encourages the model to terminate reasoning earlier when appropriate. The added term increases the logit of `</think>` relative to the average logit, making it more competitive when the model's output distribution is peaked. The term $\max(\mathbf{y}) - \text{mean}(\mathbf{y})$ reflects how peaked the model's output distribution is. This adaptive boost encourages early stopping only when the model is confident, reducing the risk of premature termination.

[5] Implementation details are provided in Appendix A.3.